Background Screening Package Builder

Technical Assessment - Senior Full-Stack Engineer

Technical Assessment

Technical Assessment for Senior Full-Stack Engineer

Overview

Build a web application that allows users to create custom background screening packages by selecting individual screening services. The system must enforce service dependencies, detect conflicts, apply pricing rules, and provide real-time feedback.

This assessment evaluates your ability to model a complex business domain, implement business rules correctly, and build an intuitive user interface that guides users through the package building process.

Technical Stack

Component	Technology
Frontend	Next.js (React)
Backend	Sinatra (Ruby)
Data Storage	Your choice: in-memory, JSON files, or simple database
Time Limit	**4 hours**
AI Assistance	Allowed and encouraged

Business Domain

Available Services

Your system should support the following screening services:

Service Name	Base Price	Category
State Criminal Search	$15.00	Criminal

Service Name	Base Price	Category
County Criminal Search	$25.00	Criminal
Federal Criminal Search	$30.00	Criminal
National Criminal Database	$40.00	Criminal
Employment Verification	$35.00	Verification
Education Verification	$35.00	Verification
Professional License Verification	$30.00	Verification
Motor Vehicle Report (MVR)	$20.00	Driving
Drug Test (5-Panel)	$45.00	Drug Screening
Drug Test (10-Panel)	$65.00	Drug Screening

Business Rules

Service Dependencies (Must be enforced)

1. County Criminal Search requires **State Criminal Search**
• Cannot add County without State already in package
2. Federal Criminal Search requires **State Criminal Search**
• Cannot add Federal without State already in package
3. National Criminal Database works as a standalone OR with other criminal searches
• No dependencies

Service Conflicts (Mutually exclusive)

1. Drug Test (10-Panel) includes everything in **Drug Test (5-Panel)**
• Cannot have both in the same package
• If 5-Panel exists and user adds 10-Panel, should replace it (or show error)

Pricing Rules

1. Volume Discount:
• 5+ services: 10% discount on total
• 8+ services: 15% discount on total
2. Category Bundle Discount:
• All 4 Criminal searches (State + County + Federal + National): Additional $20 off
• All 3 Verification services (Employment + Education + License): Additional $15 off
3. Discounts stack (volume discount + bundle discount can both apply)

Predefined Package Templates (Optional)

If time permits, you might include template packages:

Template	Services	Price
Basic Package	State Criminal + Employment Verification + Drug Test (5-Panel)	$95.00
Standard Package	State + County + Federal + National Criminal + Employment + Education Verification + Drug Test (5-Panel)	$202.50
Comprehensive Package	All services except Drug Test (5-Panel)	$254.00

Required Functionality

Core Requirements (Must Have)

1. **Display available services** with names and prices
2. **Add services to package** with real-time validation
3. **Remove services from package**
4. **Enforce dependency rules** (show clear error messages)
5. **Detect and handle conflicts** (show clear error messages)
6. **Calculate pricing** with discounts applied
7. **Show pricing breakdown:**

• Subtotal (before discounts)
• Discounts applied (itemized)
• Final total

8. **Save package** with a name
9. **Load the most recently saved package** when returning to the page
10. **Edit and re-save packages**

UI/UX Requirements

• Clean, intuitive user interface (see mockups in `assets/mockups/`)
• Real-time feedback (don't require form submission to see errors)
• Clear indication of:
• Which services are selected
• Why a service cannot be added (dependency/conflict)
• How pricing is calculated
• What discounts are applied
• Visual distinction between available, selected, disabled, and error states

Stretch Goals (Optional)

These features demonstrate additional full-stack capabilities but are not required:

Package Management:

1. View list of all saved packages with summary information
2. Load any saved package into the builder for editing
3. Delete saved packages
4. Duplicate existing packages as a starting point

Enhanced Features:

1. Package templates - Load predefined packages (Basic, Standard, Comprehensive)
2. Package comparison - View multiple packages side-by-side
3. Export functionality - Download package details as PDF or JSON
4. Search/filter services - Find services by name or category
5. Service descriptions - Tooltips or expandable details for each service
6. Undo/redo - Revert service selections

Technical Requirements

API Endpoints

At minimum, your backend should support:

```
GET    /api/services           # List all available services
GET    /api/packages/:id       # Get a specific package
POST   /api/packages           # Create/save a package
PUT    /api/packages/:id       # Update an existing package (stretch)
DELETE /api/packages/:id       # Delete a package (stretch)
GET    /api/packages           # List all packages (stretch)
POST   /api/packages/validate  # Validate a package configuration (optional)
POST   /api/packages/price     # Calculate pricing for a package (optional)
```

You may design additional endpoints as needed. The validate and price endpoints are optional - you can handle this logic entirely on frontend or backend.

Frontend Requirements

- Clean, intuitive user interface (see mockups for design guidance)
- Real-time feedback as services are selected/removed
- Clear error messaging for dependency violations and conflicts
- Responsive layout (mobile-friendly is a bonus, not required)
- Proper state management for package building

Code Quality Expectations

As a Senior Full-Stack Engineer, we're evaluating:

Area	Expectations
Architecture	Well-organized code structure with clear separation of concerns
Domain Modeling	Thoughtful representation of packages, services, rules, and pricing
Business Logic	Clean separation from presentation and data access layers
Readability	Clear naming conventions, appropriate comments where helpful
Error Handling	Graceful handling of invalid states with helpful user feedback
Testing	Some tests included demonstrating testing approach
Trade-offs	Documented decisions and rationale in README

Sample User Flows

Flow 1: Building a Basic Package

1. User sees list of available services organized by category
2. User adds "State Criminal Search" ($15)
3. User adds "Employment Verification" ($35)
4. User adds "Drug Test (5-Panel)" ($45)
5. System shows: Subtotal $95, No discounts (only 3 services), Final Total: $95
6. User enters package name "Standard Hire Package"
7. User saves package
8. System confirms save and package persists

Flow 2: Dependency Error Handling

1. User tries to add "County Criminal Search" first
2. System shows error: "County Criminal Search requires State Criminal Search to be added first"
3. County Criminal button is disabled or shows error state
4. User adds "State Criminal Search" first
5. County Criminal button becomes enabled

6. User can now successfully add "County Criminal Search"

Flow 3: Conflict Resolution

1. User adds "Drug Test (5-Panel)" ($45)
2. User later tries to add "Drug Test (10-Panel)" ($65)
3. System shows error: "Drug Test (10-Panel) conflicts with Drug Test (5-Panel) - the 10-Panel includes all 5-Panel tests"
4. User must remove 5-Panel before adding 10-Panel, or choose to replace it
5. After resolving conflict, package updates correctly

Flow 4: Maximum Discounts

1. User adds all 4 criminal searches (State, County, Federal, National)
2. User adds all 3 verification services (Employment, Education, Professional License)
3. User adds MVR
4. User has 8 services (Subtotal: $230)
5. System applies:

• 15% volume discount (8+ services): -$34.50
• Criminal bundle discount (all 4 criminal): -$20.00
• Verification bundle discount (all 3 verification): -$15.00

6. Final Total: $160.50
7. Pricing breakdown clearly shows each discount applied

Flow 5: Editing Saved Package (Stretch Goal)

1. User views list of saved packages
2. User clicks "Edit" on "Standard Hire Package"
3. Package loads into builder with all 7 services selected
4. User adds "MVR" service
5. System recalculates with 8 services, applies 15% volume discount
6. User clicks "Save Changes"
7. Package updates, user returns to package list

UI Design Reference

See the mockup images in `assets/mockups/embedded/` for visual design guidance:

Mockup	Description
`page10_img1.png`	Main Interface - Two-column layout with available services on left and current package on right

Mockup	Description
page11_img1.png	Dependency Error - Red error banner and highlighting when dependency rules are violated
page12_img1.png	Volume Discounts - Green success banner and discount badges when volume thresholds are met
page13_img1.png	Service Conflict - Yellow warning banner when conflicting services are selected
page14_img1.png	Package List (Stretch) - Grid of saved packages with edit/duplicate/delete actions

You don't need to match the mockups exactly, but they should give you a sense of:

- Layout and information hierarchy
- Color coding for different states (success/warning/error)
- What information to display at each stage
- How to indicate selected vs available services

Submission Requirements

What to Submit

1. **Source code** (GitHub repo or zip file)
2. **README** with:
- Setup instructions (how to install dependencies and run)
- How to run the application
- Any assumptions or decisions you made
- What you would add with more time
- Which stretch goals you attempted (if any)
3. **Running application** (we should be able to run it locally)

What We'll Evaluate

Criteria	What We're Looking For
Functional correctness	Does it work? Do the rules work correctly?
Code organization	Is the code well-structured and maintainable?
Domain modeling	How did you model the business domain?
User experience	Is the interface intuitive and clear?

Criteria	What We're Looking For
Edge case handling	How did you handle unusual scenarios?
Testing approach	Any tests included?
Documentation quality	Clear README and code comments where helpful
Architectural decisions	Can you explain and defend your choices?

Decision Points

As you work through this assessment, consider documenting your thinking on these architectural decisions:

1. **Where should validation logic live?** Frontend, backend, or both? Why?
2. **How will you model service dependencies?** Configuration file, code constants, database? What are the trade-offs?
3. **How will you handle the pricing calculation?** Consider maintainability if new discount rules are added.
4. **What happens when removing a service that others depend on?** (e.g., removing State Criminal when County Criminal is in the package)
5. **How will you persist package data?** What are the trade-offs of your storage choice?

Time Management Suggestions

Hour	Focus Area
Hour 1	Project setup, basic UI layout, service display, add/remove functionality
Hour 2	Dependency and conflict validation, error messaging
Hour 3	Pricing calculation, discount logic, save/load functionality
Hour 4	Polish, edge cases, testing, README documentation, stretch goals if time permits

Common Pitfalls to Avoid

- **Don't over-engineer** - Simple solutions are often better
- **Don't skimp on error handling** - Show users what went wrong and why
- **Don't forget about edge cases** - What if someone tries to remove State Criminal when County is in the package?
- **Don't ignore the domain model** - This is a key evaluation point

Notes and Tips

- **Focus on clean architecture over feature completeness** - We value quality over quantity
- **Use AI tools as you would in real work** - We want to see how you collaborate with AI
- **Document your architectural decisions** - If you make a trade-off, explain it in the README
- **Ask questions if anything is unclear** - Reach out and we'll respond. Questions are good!
- **Prioritize the core requirements first** - Get those working well before attempting stretch goals
- **There's no single "right" architecture** - We want to see *your* approach and reasoning

Appendix: Example Pricing Calculations

Example 1: Simple Package (No Discounts)

```
Services:
  - State Criminal Search: $15.00
  - Employment Verification: $35.00
  - Drug Test (5-Panel): $45.00

Subtotal: $95.00
Discounts: None (only 3 services)
═══════════════════════════════════
Total: $95.00
```

Example 2: Volume Discount (5 services)

```
Services:
  - State Criminal Search: $15.00
  - County Criminal Search: $25.00
  - National Criminal Database: $40.00
  - Employment Verification: $35.00
  - MVR: $20.00

Subtotal: $135.00
Volume Discount (10%): -$13.50
████████████████████████████████████
Total: $121.50
```

Example 3: Volume + Bundle Discounts

```
Services:
  - State Criminal Search: $15.00
  - County Criminal Search: $25.00
  - Federal Criminal Search: $30.00
  - National Criminal Database: $40.00
  - Employment Verification: $35.00
  - Education Verification: $35.00
  - Professional License Verification: $30.00
  - MVR: $20.00

Subtotal: $230.00
Volume Discount (15% for 8 services): -$34.50
Criminal Bundle Discount (all 4): -$20.00
Verification Bundle Discount (all 3): -$15.00
████████████████████████████████████
Total: $160.50
```

Example 4: Maximum Discount (All 10 Services)

```
All 10 services:
Subtotal: $340.00
Volume Discount (15% for 8+ services): -$51.00
Criminal Bundle Discount: -$20.00
Verification Bundle Discount: -$15.00
████████████████████████████████████
Total: $254.00
```

Good luck! We're excited to see what you build.